PlanMyStudy AI

AI-Powered Personalized Study Planner

Data Engineering Project Report

Author: Vishal Sreenivas

Institution: *[Your University Name]*

Course: Data Engineering

Date: March 25, 2026

Built with React · Node.js/Express · PostgreSQL · Prisma · Groq AI · YouTube Data API

Abstract

PlanMyStudy AI is an AI-powered personalized study planner that generates day-by-day course plans tailored to a learner's chosen topic, skill level, and available time. Leveraging the Groq AI inference engine (with LLaMA and Mixtral models), the platform produces structured, multi-day syllabi in JSON format. Each generated lesson is then enriched with relevant YouTube educational videos selected within a user-defined daily time budget.

The system employs a modern three-tier web architecture: a **React** single-page application communicates with a **Node.js/Express** REST API, which in turn interfaces with a **PostgreSQL** database managed through the **Prisma ORM**. Authentication is handled via **JSON Web Tokens (JWT)** with bcrypt password hashing. The backend service layer encapsulates sophisticated prompt engineering for reliable JSON generation from AI models, an exponential-backoff retry mechanism for handling API failures, ISO 8601 duration parsing for YouTube videos, and a time-budget allocation algorithm that ensures video recommendations fit the learner's daily schedule.

Key frontend features include a dashboard with a GitHub-style activity heatmap, a guided course-creation wizard with Server-Sent Events (SSE) for real-time progress feedback, a course detail view with per-day tabs, embedded video playback modals, per-lesson note-taking, quiz components, progress tracking with visual indicators, PDF export of course plans, and a course-sharing system with unique shareable links.

This report details the architecture, data flow, AI integration pipeline, database modelling, frontend design, performance characteristics, challenges encountered, and lessons learned during the development of PlanMyStudy AI.

Contents

Abstract **1**

1 Introduction **7**
1.1 Problem Statement . . . 7
1.2 Objectives . . . 7
1.3 Scope . . . 8
1.4 Technology Stack . . . 9

2 System Architecture **10**
2.1 Architectural Overview . . . 10
2.1.1 Tier 1 — Presentation Layer (Frontend) . . . 10
2.1.2 Tier 2 — Application Layer (Backend) . . . 11
2.1.3 Tier 3 — Data & External Services Layer . . . 11
2.2 Project Structure . . . 11
2.2.1 Backend Directory Layout . . . 11
2.2.2 Frontend Directory Layout . . . 12
2.3 Data Flow — Course Generation . . . 13
2.4 Authentication Flow . . . 14
2.4.1 JWT Token Lifecycle . . . 14
2.5 API Endpoint Reference . . . 15

3 Data Ingestion — AI Course Generation **16**
3.1 Overview . . . 16
3.2 Groq AI Integration . . . 16
3.2.1 Prompt Engineering . . . 16
3.3 YouTube Video Enrichment . . . 19
3.3.1 Parallel Fetching Strategy . . . 19

4 Data Transformation — Backend Service Layer **20**
4.1 Overview . . . 20
4.2 AI Response Parsing and Validation . . . 20
4.3 Retry Logic with Exponential Backoff . . . 21

4.4 ISO 8601 Duration Parsing . . . 21
4.5 Time-Budget Allocation Algorithm . . . 22
4.5.1 Video Filtering . . . 22
4.5.2 Relevance Scoring . . . 22
4.5.3 Budget-Aware Selection . . . 22

5 Data Modelling **24**
5.1 Entity–Relationship Overview . . . 24
5.2 Prisma Schema . . . 25
5.3 Model Descriptions . . . 27
5.3.1 User Model . . . 27
5.3.2 Course Model . . . 27
5.3.3 Progress Model . . . 27
5.3.4 ActivityLog Model . . . 27
5.3.5 Note Model . . . 27
5.4 Why JSON for Course Plans? . . . 28

6 Frontend & Dashboard **29**
6.1 Overview . . . 29
6.2 Landing Page . . . 29
6.3 Dashboard Page . . . 29
6.4 CreateCourse Wizard . . . 31
6.4.1 Step 1 — Topic Selection (Figure 6.4) . . . 31
6.4.2 Step 2 — Level Selection (Figure 6.5) . . . 31
6.4.3 Step 3 — Time Commitment (Figure 6.6) . . . 31
6.4.4 Step 4 — Review & Generate (Figure 6.7) . . . 32
6.5 CourseDetail Page . . . 32
6.5.1 Day Tabs . . . 33
6.5.2 Lesson Cards . . . 33
6.5.3 Video Embed Modal . . . 34
6.5.4 Quiz Section . . . 34
6.5.5 Notes . . . 34
6.5.6 Day Regeneration . . . 34
6.5.7 Progress Tracking . . . 35
6.5.8 PDF Export . . . 35
6.5.9 Course Sharing . . . 35
6.6 Component Architecture . . . 36
6.7 API Integration Layer . . . 36

7 Performance Analysis **37**

7.1 API Response Time Breakdown . . . 37
7.2 Parallel vs. Sequential YouTube Fetching . . . 37
7.3 Groq vs. OpenAI Inference Speed . . . 38
7.4 Database Performance . . . 39
7.4.1 Indexing Strategy . . . 39
7.4.2 Query Performance . . . 39
7.4.3 Connection Management . . . 39
7.5 Frontend Performance . . . 40
7.5.1 Build Optimization . . . 40
7.5.2 Runtime Optimization . . . 40
7.5.3 Render Performance . . . 40
7.6 Scalability Considerations . . . 41

8 Challenges & Solutions 42
8.1 AI JSON Malformation . . . 42
8.2 AI Model Decommissioning . . . 42
8.3 YouTube API Quota Limits . . . 42
8.4 Time-Budget Constraint Algorithm . . . 43
8.5 JWT Security Considerations . . . 43
8.6 Server-Sent Events (SSE) Reliability . . . 44
8.7 Cross-Origin Resource Sharing (CORS) . . . 44
8.8 Handling Large AI Responses . . . 44

9 Security & Testing 46
9.1 Security Architecture . . . 46
9.1.1 Authentication Security . . . 46
9.1.2 Input Validation . . . 46
9.1.3 SQL Injection Prevention . . . 47
9.1.4 Cross-Site Scripting (XSS) Prevention . . . 47
9.1.5 Rate Limiting & Error Handling . . . 47
9.2 Testing Strategy . . . 48
9.2.1 API Testing . . . 48
9.2.2 Frontend Testing . . . 48
9.2.3 AI Output Validation Testing . . . 48
9.2.4 Error Recovery Testing . . . 49

10 Conclusion 50
10.1 Summary . . . 50
10.2 Key Technical Achievements . . . 50
10.3 Lessons Learned . . . 51

10.4 Comparison with Existing Platforms 51
10.5 Future Scope 52

A Backend Code Listings **54**
A.1 Authentication Controller (authController.js) 54
A.2 JWT Utilities (jwt.js) 55
A.3 Authentication Middleware (auth.js) 56
A.4 Course Controller — Generate Course (excerpt) 56

B Sample API Payloads **58**
B.1 AI Course Generation Request 58
B.2 AI Course Generation Response (Excerpt) 58
B.3 YouTube API Response (Excerpt) 59

C Glossary **61**

References **63**

List of Figures

2.1 PlanMyStudy AI — Three-Tier System Architecture 10
2.2 End-to-End Data Flow for Course Generation 13
2.3 Authentication Flow — Registration and Login 14

5.1 Entity–Relationship Diagram for PlanMyStudy AI 24

6.1 PlanMyStudy AI — Landing Page with Hero Section and Call-to-Action Buttons 29
6.2 Dashboard Page Wireframe — Activity Heatmap, Streak Counter, and Course Cards 30
6.3 Dashboard Page — Streak Counter, Activity Calendar, Progress Statistics, and Course Cards 31
6.4 Step 1 — Topic Selection 31
6.5 Step 2 — Level Selection 31
6.6 Step 3 — Time Commitment 31
6.7 Step 4 — Review & Generate 31
6.8 CreateCourse 4-Step Wizard — Complete Course Generation Flow . . . 31

6.9 Course Detail — Header with Title, Level Badge, Progress Bar, Share and Export Buttons, and Learning Objectives . . . 32
6.10 Course Detail — Module Cards Showing Module Number, Title, Description, and Learning Goals . . . 32
6.11 Course Detail — Daily Study Plan with Day Tabs, Lesson Card, YouTube Video Resource, Quiz Prompt, Notes, and Mark as Complete Button . . 33
6.12 CourseDetail Page Wireframe — Day Tabs, Lesson Cards, Quiz, and Notes 33

7.1 Course Generation Timing Breakdown — Average Duration per Phase (7-day course) . . . 38

List of Tables

1.1 Technology Stack Overview . . . 9

2.1 REST API Endpoint Reference . . . 15

4.1 Retry Strategy Parameters . . . 21
4.2 Video Relevance Scoring Criteria . . . 22

6.1 Key Frontend Components . . . 36

7.1 Course Generation Timing Breakdown (7-day, 60 min/day) . . . 37
7.2 Parallel vs. Sequential YouTube Fetching (14 lessons) . . . 37
7.3 AI Inference Speed Comparison . . . 38
7.4 Database Index Configuration . . . 39
7.5 Estimated Scalability Metrics . . . 41

9.1 Input Validation Rules Across API Endpoints . . . 47
9.2 API Test Scenarios . . . 48

10.1 Feature Comparison with Existing Platforms . . . 52

Chapter 1

Introduction

1.1 Problem Statement

Online learning has exploded in popularity, yet learners face a paradox of choice: thousands of tutorials, courses, and videos exist for any given topic, but there is no easy way to assemble a *structured*, *time-constrained*, day-by-day study plan from these disparate resources. Students commonly:

- Spend excessive time searching for the right learning materials.
- Struggle to organize scattered resources into a coherent sequence.
- Fail to maintain consistent daily study habits due to lack of structure.
- Watch videos that are either too long or too short for their available time.

PlanMyStudy AI addresses this gap by combining generative AI with educational video curation, producing a personalized, time-aware study plan in seconds.

1.2 Objectives

The primary objectives of the PlanMyStudy AI project are:

1. **Automated Syllabus Generation**: Use AI (Groq with LLaMA/Mixtral models) to generate a multi-day, structured course plan from a single topic specification.
2. **YouTube Video Enrichment**: Automatically attach relevant educational YouTube videos to each lesson, respecting a user-defined daily time budget.
3. **Time-Budget Awareness**: Ensure that the total video duration for each day does not exceed the learner's available study time.
4. **Progress Tracking**: Allow learners to mark lessons as complete and visualize their learning journey through progress bars and activity heatmaps.

5. **Secure Multi-User Platform**: Provide JWT-based authentication so multiple users can independently generate, track, and manage their study plans.
6. **Course Sharing**: Enable users to share their generated course plans via unique public links.
7. **PDF Export**: Allow learners to download their complete course plans as formatted PDF documents.

1.3 Scope

This project encompasses:

- A full-stack web application with React frontend and Node.js/Express backend.
- Integration with the Groq AI API for course plan generation.
- Integration with the YouTube Data API v3 for video search and metadata retrieval.
- A PostgreSQL relational database with Prisma ORM for data persistence.
- JWT-based authentication and authorization.
- Real-time progress updates via Server-Sent Events (SSE).
- Activity logging with GitHub-style contribution heatmap visualization.

1.4 Technology Stack

Table 1.1: Technology Stack Overview

Layer	Technology	Purpose
Frontend	React 18	UI library for building interactive interfaces
	Vite	Fast build tool and development server
	Tailwind CSS	Utility-first CSS framework
	React Router	Client-side routing
	Axios	HTTP client for API calls
	jsPDF	PDF generation for course plan export
Backend	Node.js	JavaScript runtime environment
	Express	Minimal web framework for REST APIs
	Prisma	Type-safe ORM for PostgreSQL
	JWT (jsonwebtoken)	Token-based authentication
	bcryptjs	Password hashing with salt rounds
	Groq SDK	AI course plan generation
	Axios	HTTP client for YouTube API calls
Database	PostgreSQL	Relational database for persistent storage
External APIs	Groq AI API	LLaMA/Mixtral model inference for syllabus generation
	YouTube Data API v3	Video search, metadata, and duration retrieval

Chapter 2

System Architecture

2.1 Architectural Overview

PlanMyStudy AI follows a **three-tier web architecture** consisting of a client-side presentation layer, a server-side application layer, and a data/external-services layer. This separation ensures clean modularity, independent scalability of each tier, and clear boundaries for testing and deployment.

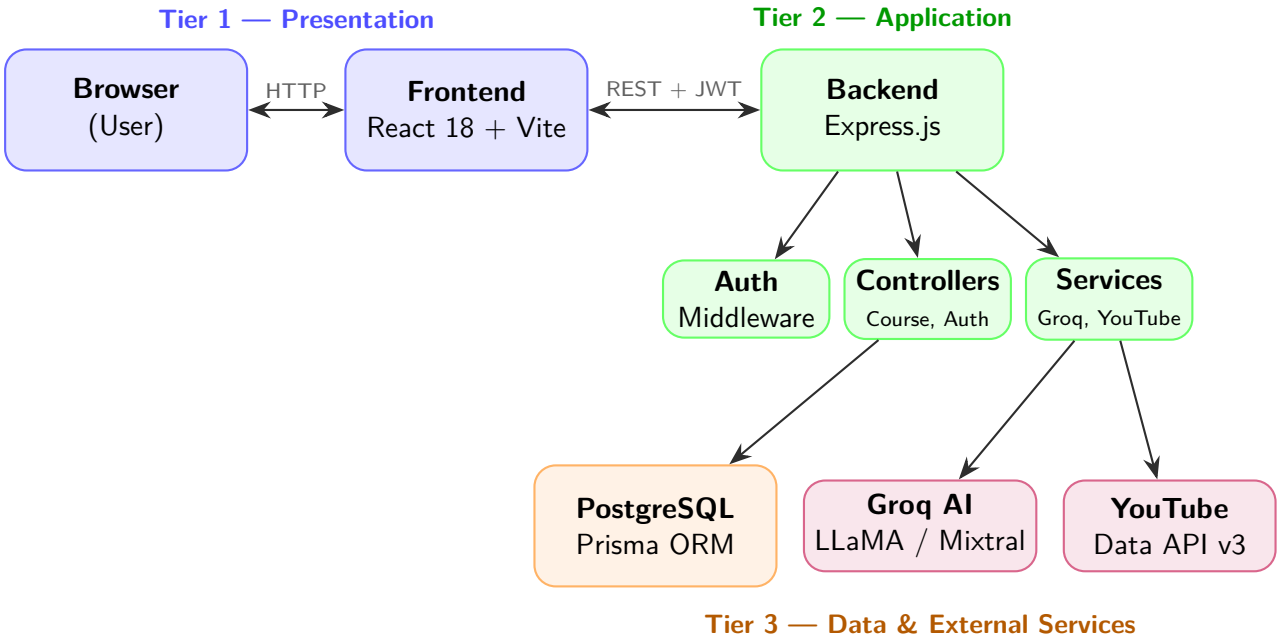

Figure 2.1: PlanMyStudy AI — Three-Tier System Architecture

2.1.1 Tier 1 — Presentation Layer (Frontend)

The React single-page application handles all user interaction. Built with Vite for fast development and optimized production builds, it uses React Router for client-side nav-

igation and Axios with interceptors for API communication. The frontend stores JWT tokens in `localStorage` and automatically attaches them to every outgoing request via an Axios request interceptor.

2.1.2 Tier 2 — Application Layer (Backend)

The Express server exposes a RESTful API organized into authentication routes and course management routes. It follows the **Controller–Service–Middleware** pattern:

- **Controllers** handle HTTP request/response logic and input validation.
- **Services** encapsulate business logic and external API integration (Groq AI, YouTube).
- **Middleware** provides cross-cutting concerns such as JWT authentication.

2.1.3 Tier 3 — Data & External Services Layer

This tier comprises the PostgreSQL database (accessed through Prisma ORM) and two external APIs:

1. **Groq AI API**: Provides ultra-fast LLM inference for generating structured course plans.
2. **YouTube Data API v3**: Supplies video search results and content metadata including ISO 8601 durations.

2.2 Project Structure

2.2.1 Backend Directory Layout

```
backend/
          prisma/
                schema.prisma          # Database schema definition
          src/
                config/
                      database.js        # Prisma client initialization
                      env.js             # Environment variable validation
                controllers/
                      authController.js  # Authentication logic
                      courseController.js # Course management logic
                      activityController.js # Activity tracking
                      noteController.js  # Note management
                middleware/
                      auth.js            # JWT authentication middleware
```

```
                routes/
                      authRoutes.js       # Auth endpoints
                      courseRoutes.js      # Course endpoints
                      activityRoutes.js   # Activity endpoints
                      noteRoutes.js       # Note endpoints
                services/
                      groqService.js      # AI course generation
                      youtubeService.js   # YouTube video fetching
                utils/
                      errors.js           # Error handling utilities
                      jwt.js              # JWT token utilities
                server.js               # Express app entry point
          package.json
```

Listing 2.1: Backend Project Structure

2.2.2 Frontend Directory Layout

```
frontend/
          src/
                components/
                      Layout.jsx          # Page layout wrapper
                      ProtectedRoute.jsx # Route guard
                pages/
                      Home.jsx            # Landing page
                      Login.jsx           # Login page
                      Register.jsx        # Registration page
                      Dashboard.jsx       # Course list + activity heatmap
                      CreateCourse.jsx    # Course generation wizard
                      CourseDetail.jsx    # Course view with day tabs
                services/
                      api.js              # API client with interceptors
                utils/
                      auth.js             # Auth helper functions
                      pdfExport.js        # PDF generation utility
                App.jsx                 # Main app with routing
                main.jsx                # React entry point
                index.css               # Global styles (Tailwind)
          tailwind.config.js
          vite.config.js
          package.json
```

Listing 2.2: Frontend Project Structure

2.3 Data Flow — Course Generation

The end-to-end data flow for generating a new course is the most complex pipeline in the system. Figure 2.2 illustrates the complete sequence.

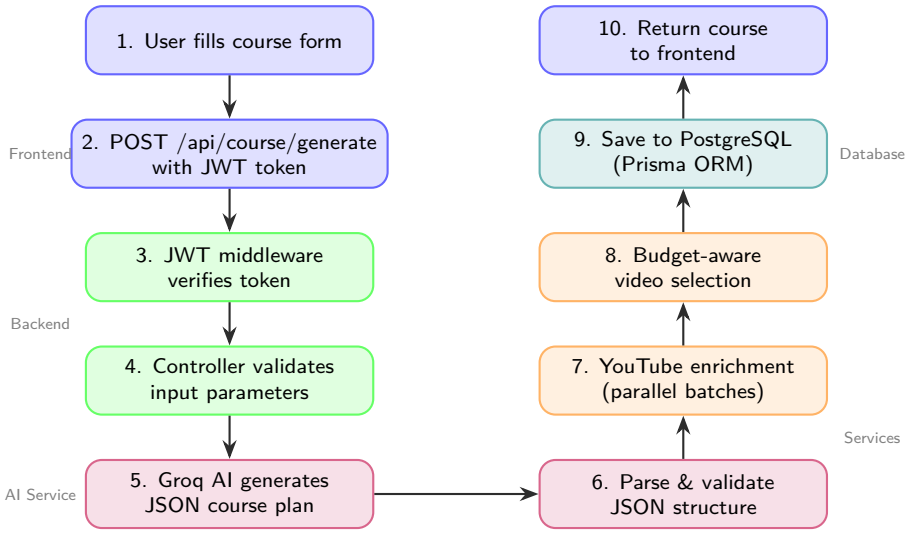

Figure 2.2: End-to-End Data Flow for Course Generation

1. **User Input**: The user fills in topic, skill level, number of days, and minutes per day on the `CreateCourse` page.
2. **API Request**: The frontend sends a `POST /api/course/generate` request with the JWT token.
3. **Authentication**: The `authenticate` middleware verifies the JWT and attaches `userId` to the request.
4. **Input Validation**: The `generateCourse` controller validates all fields (level must be beginner/intermediate/advanced, days 1–365, time 15–480 minutes).
5. **AI Generation**: The `groqService.generateCoursePlan()` function constructs a structured prompt and calls the Groq API. On failure, it retries up to 3 times with exponential backoff, trying multiple models sequentially.
6. **JSON Validation**: The AI response is parsed and validated for required fields (`overview`, `modules`, `dailyPlan`).
7. **YouTube Enrichment**: The `youtubeService.enrichCourseWithVideos()` function fetches videos for each lesson in parallel batches of 5, then sequentially selects the best video per lesson respecting the remaining daily time budget.
8. **Database Storage**: The complete course plan (including video metadata) is serialized as JSON and stored in the `planJson` column of the `Course` table.
9. **Response**: The course data is returned to the frontend for display.

2.4 Authentication Flow

Authentication is a critical cross-cutting concern that touches every protected endpoint. Figure 2.3 shows the complete registration and login flow.

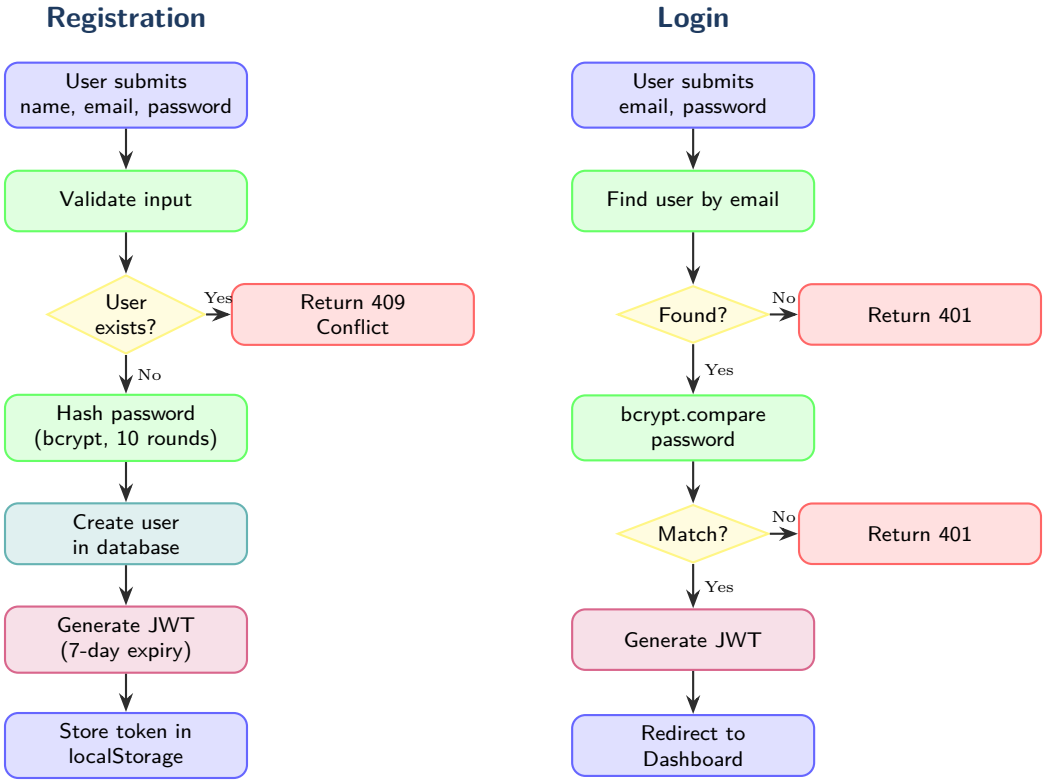

Figure 2.3: Authentication Flow — Registration and Login

2.4.1 JWT Token Lifecycle

Tokens follow a well-defined lifecycle within the system:

1. **Generation**: Upon successful registration or login, the server signs a JWT containing the `userId` claim with a secret key, setting a 7-day expiration.
2. **Storage**: The frontend stores the token in `localStorage` for persistence across page refreshes.
3. **Transmission**: Every API request includes the token in the `Authorization: Bearer <token>` header, added automatically by the Axios request interceptor.
4. **Verification**: The `authenticate` middleware extracts the token, verifies its signature and expiration using `jsonwebtoken.verify()`, and attaches the decoded `userId` to the request object.
5. **Expiration**: After 7 days, the token becomes invalid. The Axios response interceptor catches the resulting 401 error, clears the stored token, and redirects the user to the login page.

2.5 API Endpoint Reference

Table 2.1 lists all REST API endpoints exposed by the backend.

Table 2.1: REST API Endpoint Reference

Method	Endpoint	Auth	Description
POST	`/api/auth/register`	No	Register a new user
POST	`/api/auth/login`	No	Login and receive JWT
POST	`/api/course/generate`	Yes	Generate a new AI course
POST	`/api/course/generate-sse`	Yes	Generate course with SSE progress
GET	`/api/course/`	Yes	List user's courses
GET	`/api/course/:id`	Yes	Get course by ID
DELETE	`/api/course/:id`	Yes	Delete a course
POST	`/api/course/:id/regenerate`	Yes	Regenerate a single day
POST	`/api/course/:id/share`	Yes	Toggle course sharing
GET	`/api/course/:id/sharing`	Yes	Get sharing status
GET	`/api/course/shared/:token`	No	View shared course (public)
PUT	`/api/course/progress`	Yes	Update lesson progress
GET	`/api/activity/`	Yes	Get activity log

Chapter 3

Data Ingestion — AI Course Generation

3.1 Overview

In the context of PlanMyStudy AI, "data ingestion" refers to the process of generating and acquiring structured educational content through AI inference and external API calls. Unlike traditional data engineering pipelines that ingest data from files, databases, or streams, this system *creates* data on demand by prompting a large language model and then enriching the output with video metadata from the YouTube Data API.

3.2 Groq AI Integration

The system uses the Groq inference platform, which provides ultra-fast inference speeds using custom hardware (Language Processing Units). The service supports multiple models with automatic fallback:

1. `llama-3.1-70b-versatile` — Primary model (highest quality)
2. `llama-3.1-8b-instant` — Fallback (faster, slightly lower quality)
3. `mixtral-8x7b-32768` — Secondary fallback (mixture-of-experts architecture)

3.2.1 Prompt Engineering

The AI prompt is carefully structured to produce predictable, parseable JSON output. Key design decisions include:

- **System message**: Enforces JSON-only responses with no markdown formatting.

- **Template embedding**: The exact JSON schema is included in the prompt so the model knows the expected structure.
- **Constraint specification**: Total days, minutes per day, lessons per day, and module count are explicitly stated.
- **Quiz generation**: Each lesson includes 2–3 multiple-choice questions with explanations.
- **Importance tagging**: Lessons are tagged as `"core"`, `"important"`, or `"bonus"` to guide prioritization.

```
export const generateCoursePlan = async (topic, level, days,
    timePerDay) => {
  const maxRetries = 3;
  let attempt = 0;

  while (attempt < maxRetries) {
    try {
      const prompt = `Create a ${days}-day study plan for "${topic}"
        (${level} level, ${timePerDay} min/day).

Return JSON only:
{
  "overview": {
    "title": "Course Title",
    "description": "Brief description",
    "objectives": ["obj1", "obj2", "obj3"]
  },
  "modules": [
    {"id": "m1", "title": "Module Name",
     "description": "Brief", "objectives": ["obj"]}
  ],
  "dailyPlan": [
    {
      "day": 1,
      "moduleId": "m1",
      "lessons": [
        {
          "id": "l1", "title": "Lesson Title",
          "description": "Brief description",
          "objectives": ["obj"],
          "timeMinutes": ${Math.floor(timePerDay / 2)},
          "keywords": ["keyword1", "keyword2"],
          "importance": "core",
          "quiz": [
            {
              "question": "Quiz question",
```

```
                "options": ["A) ...", "B) ...", "C) ...", "D) ..."],
                "correctAnswer": "A",
                "explanation": "Why A is correct"
              }
            ]
          }
        ]
      }
    ]
}`;
        // Call Groq API with 30-second timeout
        const models = ['llama-3.1-70b-versatile',
                        'llama-3.1-8b-instant',
                        'mixtral-8x7b-32768'];
        let response;
        for (const model of models) {
          try {
            response = await groq.chat.completions.create({
              model: model,
              messages: [
                { role: 'system',
                  content: 'You are an expert educational course designer.
                    Always return valid JSON only.' },
                { role: 'user', content: prompt }
              ],
              temperature: 0.7,
              max_tokens: 3000,
            });
            break; // Success
          } catch (modelError) {
            if (modelError.message?.includes('decommissioned'))
              continue; // Try next model
            else throw modelError;
          }
        }
        // Parse and validate JSON response ...
      } catch (error) {
        attempt++;
        if (attempt >= maxRetries) throw new AppError(errorMessage, 500);
        // Exponential backoff
        await new Promise(r => setTimeout(r, 1000 * attempt));
      }
    }
};
```

Listing 3.1: Groq AI Course Generation (groqService.js – excerpt)

3.3 YouTube Video Enrichment

After the AI generates the course plan, each lesson is enriched with a relevant YouTube video. The enrichment pipeline proceeds as follows:

1. **Query Construction**: For each lesson, a search query is built from the lesson title, up to 2 unique keywords, and the word "tutorial".
2. **YouTube Search**: The YouTube Data API `search.list` endpoint is called with `videoCategoryId=27` (Education), `videoEmbeddable=true`, and `safeSearch=strict`.
3. **Duration Retrieval**: A second API call to `videos.list` fetches `contentDetails` (ISO 8601 duration), `statistics` (view count), and `snippet` (metadata) for each result.
4. **Video Selection**: The best video is chosen using a multi-criteria scoring algorithm (Section 4.5).

3.3.1 Parallel Fetching Strategy

To minimize total enrichment time, the system fetches videos for all lessons within a day in parallel using `Promise.all`, processing them in batches of 5 to avoid YouTube API rate limits:

```
const fetchVideosInBatches = async (lessons, courseTopic, batchSize
    = 5) => {
  const results = [];
  for (let i = 0; i < lessons.length; i += batchSize) {
    const batch = lessons.slice(i, i + batchSize);
    // Fetch all videos in this batch in parallel
    const batchResults = await Promise.all(
      batch.map(lesson => fetchVideosForLesson(lesson, courseTopic))
    );
    results.push(...batchResults);
    // Small delay between batches to avoid rate limiting
    if (i + batchSize < lessons.length) {
      await new Promise(resolve => setTimeout(resolve, 100));
    }
  }
  return results;
};
```

Listing 3.2: Parallel Video Fetching (youtubeService.js – excerpt)

Chapter 4

Data Transformation — Backend Service Layer

4.1 Overview

The "transformation" stage in PlanMyStudy AI corresponds to the backend service layer, which transforms raw AI output and YouTube API responses into validated, enriched, and budget-aware course plans. This chapter details the key transformation algorithms and validation logic.

4.2 AI Response Parsing and Validation

The Groq AI model occasionally returns responses wrapped in markdown code blocks or with minor structural deviations. The parsing pipeline handles these cases:

```
// Remove markdown code blocks if present
let jsonString = content;
if (content.startsWith('''')) {
  jsonString = content
    .replace(/```json\n?/g, '')
    .replace(/```\n?/g, '')
    .trim();
}

// Parse JSON
const coursePlan = JSON.parse(jsonString);

// Validate structure
if (!coursePlan.overview || !coursePlan.modules || !coursePlan.
    dailyPlan) {
```

```
  throw new Error('Invalid course plan structure');
}

// Validate daily plan matches requested days
if (coursePlan.dailyPlan.length !== days) {
  throw new Error(
    `Daily plan has ${coursePlan.dailyPlan.length} days, expected ${days}`
  );
}
```

Listing 4.1: JSON Parsing with Markdown Stripping

4.3 Retry Logic with Exponential Backoff

The system implements a robust retry mechanism for both AI generation and YouTube API calls:

Table 4.1: Retry Strategy Parameters

Parameter	AI (Groq)	YouTube
Maximum Retries	3	1 (per lesson)
Backoff Strategy	Exponential (1000 × attempt ms)	Fixed 100 ms between batches
Rate Limit Wait	10 000 ms	N/A
Timeout	30 000 ms	5 000 ms per lesson
Model Fallback	Yes (3 models)	N/A

4.4 ISO 8601 Duration Parsing

YouTube returns video durations in ISO 8601 format (e.g., `PT4M13S` for 4 minutes and 13 seconds). The system parses these into seconds for budget calculations:

```
const parseISO8601Duration = (duration) => {
  if (!duration) return 0;
  const match = duration.match(/PT(?:(\d+)H)?(?:(\d+)M)?(?:(\d+)S)?/);
  if (!match) return 0;
  const hours = parseInt(match[1] || 0, 10);
  const minutes = parseInt(match[2] || 0, 10);
  const seconds = parseInt(match[3] || 0, 10);
  return hours * 3600 + minutes * 60 + seconds;
};
```

Listing 4.2: ISO 8601 Duration Parser

4.5 Time-Budget Allocation Algorithm

The most sophisticated transformation in the pipeline is the time-budget-aware video selection. For each day, the system maintains a remaining time budget and selects videos that fit within it.

4.5.1 Video Filtering

Before scoring, videos are filtered through multiple criteria:

- **Blacklist Filter**: Videos with titles containing keywords like "full course", "crash course", "bootcamp", "masterclass", "10 hours", etc. are excluded.
- **Minimum Duration**: Videos shorter than 60 seconds are excluded (likely intros or Shorts).
- **Maximum Duration**: Videos longer than 1.5× the lesson's estimated time are excluded.

4.5.2 Relevance Scoring

Each candidate video receives a relevance score (0–100) based on:

Table 4.2: Video Relevance Scoring Criteria

Criterion	Score
Exact lesson title match in video title	+40
Keyword match in video title (per keyword)	+15
Keyword match in video description (per keyword)	+5
Educational indicator in title (tutorial, etc.)	+5
Title length > 100 characters (penalty)	−10

4.5.3 Budget-Aware Selection

After scoring, the algorithm selects the highest-scoring video that fits within the remaining daily budget (with a ±2-minute buffer). If no video fits, the shortest relevant video is selected as a fallback and flagged with `withinBudget: false`.

```
const selectBestVideo = (videos, lesson, remainingBudgetSeconds) =>
    {
  // ... filtering and scoring logic ...
  // Sort by relevance score (descending)
  candidates.sort((a, b) => b.relevanceScore - a.relevanceScore);
  // Find the best video that fits the remaining budget
```

```
  const budgetWithBuffer = remainingBudgetSeconds + (TIME_BUFFER_MINUTES * 60);
  for (const video of candidates) {
    if (video.durationSeconds <= budgetWithBuffer) {
      return { ...video, withinBudget: true };
    }
  }
  // Fallback: pick the shortest relevant video
  const shortestVideo = candidates.reduce((shortest, current) =>
    current.durationSeconds < shortest.durationSeconds ? current : shortest
  );
  return {
    ...shortestVideo,
    withinBudget: false,
    budgetExceededBy: shortestVideo.durationSeconds - remainingBudgetSeconds,
  };
};
```

Listing 4.3: Budget-Aware Video Selection (excerpt)

Chapter 5

Data Modelling

5.1 Entity–Relationship Overview

PlanMyStudy AI uses a PostgreSQL relational database with five models managed through the Prisma ORM. The entity relationships are illustrated in Figure 5.1.

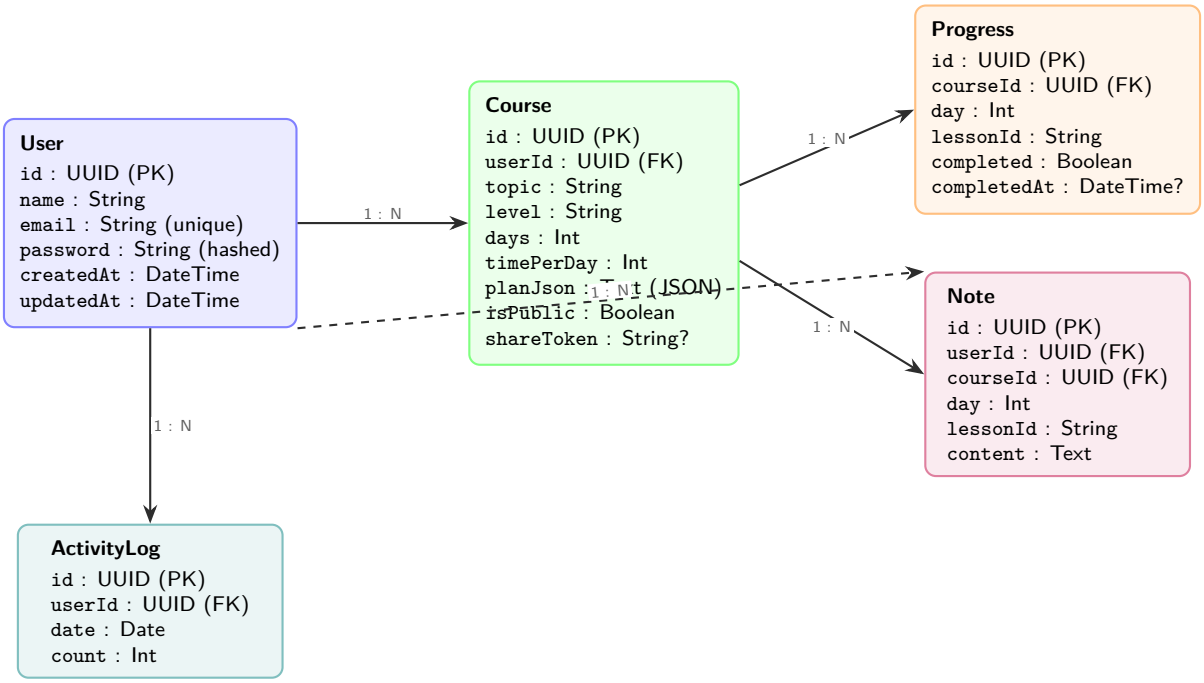

Figure 5.1: Entity–Relationship Diagram for PlanMyStudy AI

```
User (1) < (Many) Course

                        (1)
                      < (Many) Progress
                      < (Many) Note
```

```
User (1) < (Many) ActivityLog
```

5.2 Prisma Schema

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id         String   @id @default(uuid())
  name       String
  email      String   @unique
  password   String   // Hashed with bcrypt (10 rounds)
  createdAt  DateTime @default(now())
  updatedAt  DateTime @updatedAt
  courses    Course[]
  activities ActivityLog[]
  notes      Note[]
}

model Course {
  id          String   @id @default(uuid())
  userId      String
  topic       String
  level       String   // beginner | intermediate | advanced
  days        Int
  timePerDay  Int      // minutes per day
  planJson    String   @db.Text  // Full course plan as JSON
  isPublic    Boolean  @default(false)
  shareToken  String?  @unique
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  user        User     @relation(fields: [userId],
                                 references: [id],
                                 onDelete: Cascade)
  progress    Progress[]
  notes       Note[]
  @@index([userId])
  @@index([shareToken])
```

```
}

model Progress {
  id          String    @id @default(uuid())
  courseId     String
  day         Int
  lessonId    String
  completed   Boolean   @default(false)
  completedAt DateTime?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  course      Course    @relation(fields: [courseId],
                                  references: [id],
                                  onDelete: Cascade)
  @@unique([courseId, day, lessonId])
  @@index([courseId])
}

model ActivityLog {
  id        String   @id @default(uuid())
  userId    String
  date      DateTime @db.Date
  count     Int      @default(1)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  user      User     @relation(fields: [userId],
                                references: [id],
                                onDelete: Cascade)
  @@unique([userId, date])
  @@index([userId])
  @@index([date])
}

model Note {
  id        String   @id @default(uuid())
  userId    String
  courseId   String
  day       Int
  lessonId  String
  content   String   @db.Text
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  user      User     @relation(fields: [userId],
                                references: [id],
                                onDelete: Cascade)
  course    Course   @relation(fields: [courseId],
                                references: [id],
```

```
                                    onDelete: Cascade)
  @@unique([courseId, day, lessonId])
  @@index([userId])
  @@index([courseId])
}
```

Listing 5.1: Complete Prisma Schema (schema.prisma)

5.3 Model Descriptions

5.3.1 User Model

The `User` model stores authentication credentials and serves as the root entity. Passwords are stored as bcrypt hashes with 10 rounds of salting. UUIDs are used as primary keys for unpredictability and distributed-system readiness.

5.3.2 Course Model

The `Course` model stores both metadata (topic, level, days, timePerDay) and the full AI-generated course plan as a JSON text field (`planJson`). This model also supports course sharing through an optional unique `shareToken` and an `isPublic` boolean flag.

5.3.3 Progress Model

The `Progress` model tracks per-lesson completion using a composite unique constraint on `(courseId, day, lessonId)`. This uses the Prisma `upsert` operation to create or update a progress record atomically.

5.3.4 ActivityLog Model

The `ActivityLog` model tracks daily user activity counts with a composite unique constraint on `(userId, date)`. This data powers the GitHub-style activity heatmap on the dashboard.

5.3.5 Note Model

The `Note` model stores per-lesson markdown notes with a composite unique constraint on `(courseId, day, lessonId)`, allowing one note per lesson per user.

5.4 Why JSON for Course Plans?

The course plan is stored as a JSON text field rather than normalized relational tables for several reasons:

1. **Dynamic Structure**: AI generates plans with varying structures depending on the topic.
2. **Atomic Read/Write**: The entire plan is always read and written as a unit, making JSON storage efficient.
3. **Schema Flexibility**: New fields (quizzes, importance tags, video stats) can be added without database migrations.
4. **Performance**: A single database read retrieves the complete plan, avoiding complex joins across lesson/module/day tables.
5. **Simplicity**: The JSON structure mirrors the API response format, eliminating serialization overhead.

Chapter 6

Frontend & Dashboard

6.1 Overview

The PlanMyStudy AI frontend is a React 18 single-page application built with Vite and styled with Tailwind CSS. It provides a rich, interactive experience for creating courses, tracking progress, and consuming educational content—all within the browser. The application uses Framer Motion for smooth page transitions and micro-animations, React Hot Toast for notification feedback, and a comprehensive skeleton loading system for perceived performance.

6.2 Landing Page

The landing page (Figure 6.1) is the first screen users encounter. It features a dark-themed hero section with the tagline "Design your learning path **with intelligence**" in a bold typographic treatment with an accent gradient on the key phrase. Two prominent call-to-action buttons—"Create New Course" and "Dashboard"—guide users into the application. The navigation bar displays the PlanMyStudy AI logo, links to Dashboard and Create Course, and the authenticated user's name with a logout button.

Figure 6.1: PlanMyStudy AI — Landing Page with Hero Section and Call-to-Action Buttons

6.3 Dashboard Page

The dashboard serves as the user's home screen after login. It features:

- **Activity Heatmap**: A GitHub-style contribution calendar showing daily study

activity over the past year. Each cell is colour-coded based on the number of lessons completed that day, pulled from the `ActivityLog` model.

- **Streak Counter**: Displays the user's current consecutive-day learning streak, motivating daily engagement.
- **Course Progress Bar**: An aggregate progress indicator showing overall completion across all courses.
- **Course Cards**: A responsive grid of cards (1 column on mobile, 2 on tablet, 3 on desktop) showing each generated course with topic, level, day count, and a progress bar indicating completion percentage.
- **Quick Actions**: A prominent "Create New Course" button with hover and tap animations.
- **Empty State**: When no courses exist, a custom illustration with a call-to-action guides new users.
- **Skeleton Loading**: While data loads, animated skeleton placeholders maintain layout stability.

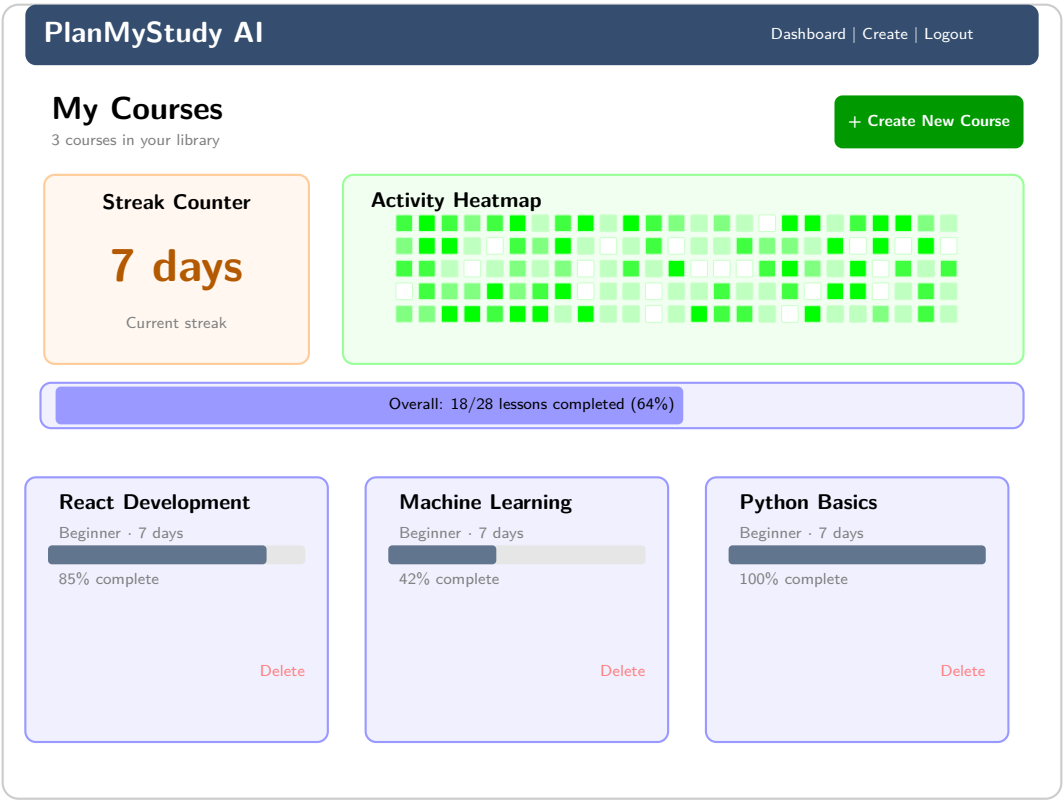

Figure 6.2: Dashboard Page Wireframe — Activity Heatmap, Streak Counter, and Course Cards

Figure 6.3 shows the actual dashboard as rendered in production, displaying the streak counter, activity calendar heatmap, overall progress bar with statistics cards (Completed, In Progress, Total Courses, Lessons), and course cards with progress indicators.

Figure 6.3: Dashboard Page — Streak Counter, Activity Calendar, Progress Statistics, and Course Cards

6.4 CreateCourse Wizard

The course creation page provides a guided 4-step wizard that walks users through the entire course specification process. Each step has a progress bar at the top showing completion percentage. Figures 6.4 through 6.7 show the four steps.

Figure 6.4: Step 1 — Topic Selection

Figure 6.5: Step 2 — Level Selection

Figure 6.6: Step 3 — Time Commitment

Figure 6.7: Step 4 — Review & Generate

Figure 6.8: CreateCourse 4-Step Wizard — Complete Course Generation Flow

6.4.1 Step 1 — Topic Selection (Figure 6.4)

The first step presents a text input field for the learning topic with a list of popular topic suggestions (React Development, Machine Learning, Python Programming, Data Science, Web Design, etc.) as clickable chips. A "Pro tip" banner advises users to be specific with their topic for better results.

6.4.2 Step 2 — Level Selection (Figure 6.5)

Three level options are presented as detailed cards:

- **Complete Beginner**: "Start with fundamentals, gentle learning curve, more practice exercises."
- **Some Experience**: "Skip basic concepts, focus on practical skills, intermediate challenges."
- **Advanced Learner**: "Advanced concepts only, complex projects, expert-level material."

Each card shows a distinctive icon and is highlighted with a cyan border when selected.

6.4.3 Step 3 — Time Commitment (Figure 6.6)

Four preset schedule options are offered as cards:

- **Quick Intro**: 3 days × 30 min/day
- **One Week Intensive**: 7 days × 60 min/day (highlighted as "Popular choice")
- **Two Week Journey**: 14 days × 45 min/day
- **Monthly Challenge**: 30 days × 30 min/day

Below the presets, a "Or customize your plan" section allows manual entry of total days (1–365) and minutes per day (15–480), with a calculated total study time displayed.

6.4.4 Step 4 — Review & Generate (Figure 6.7)

The final step displays a summary card titled "Your Learning Plan" showing the selected topic, level, and duration. A "What to expect" section lists the features the user will receive:

- Personalized daily lessons tailored to their level
- Curated video resources and learning materials
- Interactive quizzes and progress tracking
- Notes feature to capture insights

The "Generate My Course" button triggers the AI generation pipeline, transitioning to SSE progress updates:

- "Creating your personalized study plan…"
- "Study plan created! Finding videos for Day 1…"
- "Finding videos for Day 2…" (and so on)
- "All videos found! Saving your course…"
- "Course created successfully!"

6.5 CourseDetail Page

The course detail view is the most feature-rich page in the application, containing multiple interactive sections for a comprehensive learning experience. Figure 6.9 shows the course overview header with title, learning objectives, and action buttons. Figure 6.10 displays the module cards, and Figure 6.11 shows the daily study plan with a lesson card, video resource, quiz prompt, and notes section.

Figure 6.9: Course Detail — Header with Title, Level Badge, Progress Bar, Share and Export Buttons, and Learning Objectives

Figure 6.10: Course Detail — Module Cards Showing Module Number, Title, Description, and Learning Goals

Figure 6.11: Course Detail — Daily Study Plan with Day Tabs, Lesson Card, YouTube Video Resource, Quiz Prompt, Notes, and Mark as Complete Button

Figure 6.12 provides a schematic layout of all these components together.

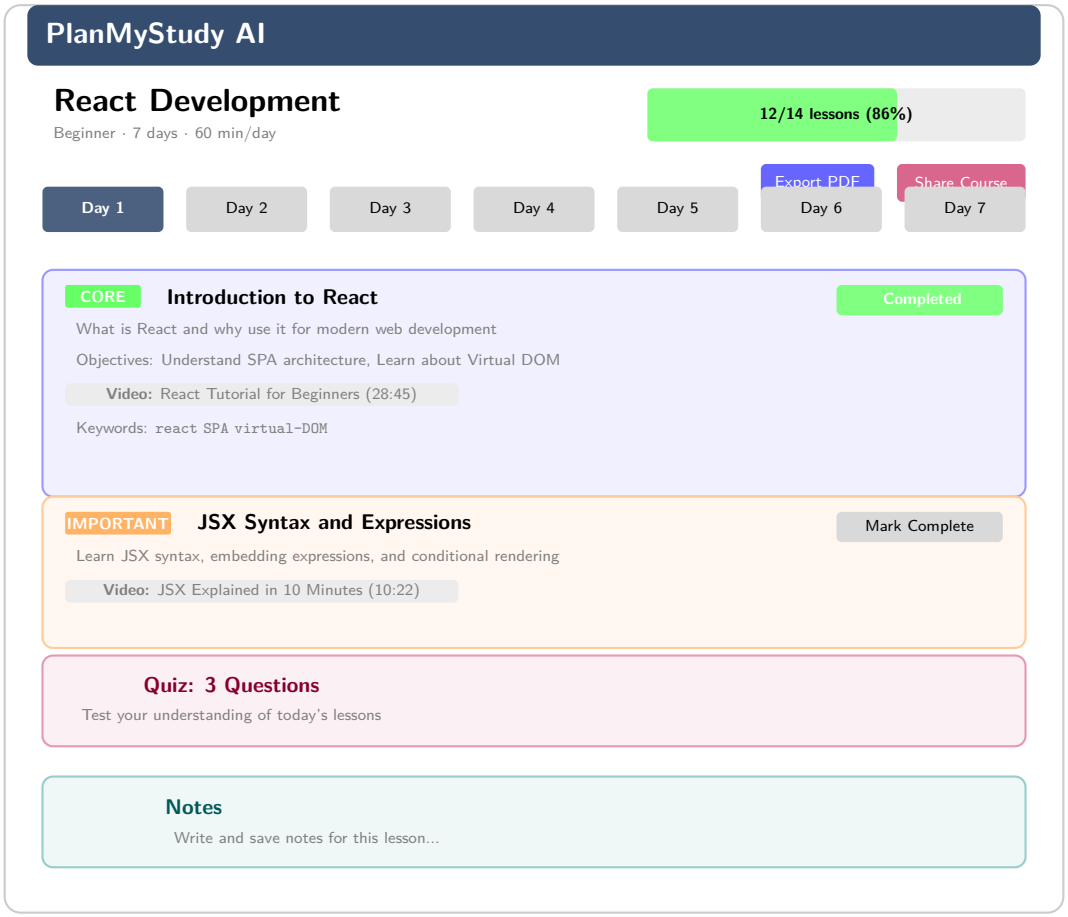

Figure 6.12: CourseDetail Page Wireframe — Day Tabs, Lesson Cards, Quiz, and Notes

6.5.1 Day Tabs

A horizontal tab bar at the top allows switching between days. Each tab shows the day number and a completion indicator (checkmark when all lessons for that day are complete, or an open circle otherwise). The active tab is highlighted with the primary colour.

6.5.2 Lesson Cards

Each day displays its lessons as expandable cards showing:

- Lesson title, description, and estimated time in minutes
- Learning objectives as a bulleted list

- Importance badge with colour coding: **CORE** (green), **IMPORTANT** (orange), **BONUS** (blue)
- A "Mark Complete" toggle button that transitions to a green checkmark
- Keywords displayed as styled tags for quick topic identification

6.5.3 Video Embed Modal

Each lesson with a video resource shows a YouTube thumbnail with duration overlay. Clicking it opens a full-screen modal with:

- An embedded YouTube player (`<iframe>`) for in-page playback
- Video title, channel name, and formatted duration
- A budget indicator showing whether the video fits within the daily time budget
- Direct link to watch on YouTube

The `videoEmbeddable=true` and `videoSyndicated=true` filters in the YouTube API ensure all returned videos can be embedded.

6.5.4 Quiz Section

Each lesson includes 2–3 multiple-choice quiz questions generated by the AI. The quiz component provides:

- Four options per question with radio-button selection
- Immediate visual feedback (green for correct, red for incorrect)
- Explanation text revealed after answering
- Score summary at the end of all questions

6.5.5 Notes

A per-lesson markdown note editor allows users to take and save notes during their study session. Notes are persisted in the `Note` model via `PUT /api/notes` with a unique constraint on `(courseId, day, lessonId)`, ensuring one note per lesson.

6.5.6 Day Regeneration

If a user is unsatisfied with a particular day's content, they can click "Regenerate Day" to trigger a fresh AI generation for just that day. The backend calls `regenerateSingleDay()` which:

1. Extracts module context from the existing plan
2. Prompts the AI with higher temperature (0.8) for more variety
3. Enriches the regenerated day with new YouTube videos

4. Replaces only the target day in the stored `planJson`

6.5.7 Progress Tracking

- **Per-Lesson**: Checkbox toggles that call `PUT /api/course/progress` to upsert completion records. Each toggle triggers an activity log update.
- **Per-Day**: A progress bar showing `completed / total` lessons for the current day.
- **Overall**: A progress indicator at the top of the page showing total course completion percentage across all days.
- **Visual Feedback**: Completed lessons receive a green checkmark, completed days show a checkmark in their tab, and the activity heatmap on the dashboard updates in real time.

6.5.8 PDF Export

The jsPDF library generates a formatted PDF document containing the full course plan, including:

- Course title, level, and duration metadata
- All days with their lessons, objectives, and time estimates
- Video links (as clickable URLs in the PDF)
- Completion status for tracking

6.5.9 Course Sharing

Users can generate a unique public URL for any course via `POST /api/course/:id/share`. Shared courses are accessible without authentication at `/shared/:token`, showing the course plan in read-only mode with the creator's name.

6.6 Component Architecture

Table 6.1: Key Frontend Components

Component	Type	Responsibility
`App.jsx`	Router	Defines all routes and wraps protected routes
`Layout.jsx`	Wrapper	Header, navigation, and page container
`ProtectedRoute.jsx`	Guard	Redirects unauthenticated users to login
`Home.jsx`	Page	Landing page with feature highlights
`Login.jsx`	Page	Email/password login form
`Register.jsx`	Page	Registration form with validation
`Dashboard.jsx`	Page	Course list and activity heatmap
`CreateCourse.jsx`	Page	Course creation wizard with SSE progress
`CourseDetail.jsx`	Page	Course view with day tabs, videos, progress
`api.js`	Service	Axios instance with JWT interceptor
`pdfExport.js`	Utility	jsPDF-based course plan export

6.7 API Integration Layer

The frontend uses a centralized Axios instance (`services/api.js`) with:

- **Request Interceptor**: Automatically attaches the JWT token from `localStorage` to every outgoing request's `Authorization` header.
- **Response Interceptor**: Intercepts 401 responses globally, clears the stored token, and redirects to the login page.
- **Base URL Configuration**: Reads from environment variables for seamless deployment switching between development and production.

Chapter 7

Performance Analysis

7.1 API Response Time Breakdown

Course generation is the most latency-sensitive operation in the system. Table 7.1 shows the typical timing breakdown for generating a 7-day course with 60 minutes per day.

Table 7.1: Course Generation Timing Breakdown (7-day, 60 min/day)

Phase	**Duration (ms)**	**Percentage**
Input Validation	∼5	<1%
Groq AI Plan Generation	∼2,000–4,000	20–35%
JSON Parsing & Validation	∼10	<1%
YouTube Video Enrichment	∼5,000–8,000	50–65%
Database Save (Prisma)	∼50–100	<2%
Total (Typical)	**8,000–12,000**	**100%**

7.2 Parallel vs. Sequential YouTube Fetching

One of the key performance optimizations was switching from sequential to parallel YouTube API calls. Table 7.2 compares the two approaches.

Table 7.2: Parallel vs. Sequential YouTube Fetching (14 lessons)

Strategy	**Total Time (ms)**	**API Calls**
Sequential (1 at a time)	∼14,000	28 (search + details per lesson)
Parallel (batch of 5)	∼4,200	28 (same, but concurrent)
Speedup	**∼3.3× faster**	

The parallel batching strategy (batch size = 5) was chosen as the optimal balance between speed and YouTube API rate limit compliance. Each batch makes up to 10 API calls (5 search + 5 details), with a 100 ms delay between batches.

Figure 7.1 visualizes the timing breakdown across the four major phases of the pipeline, clearly illustrating that YouTube enrichment dominates the total latency.

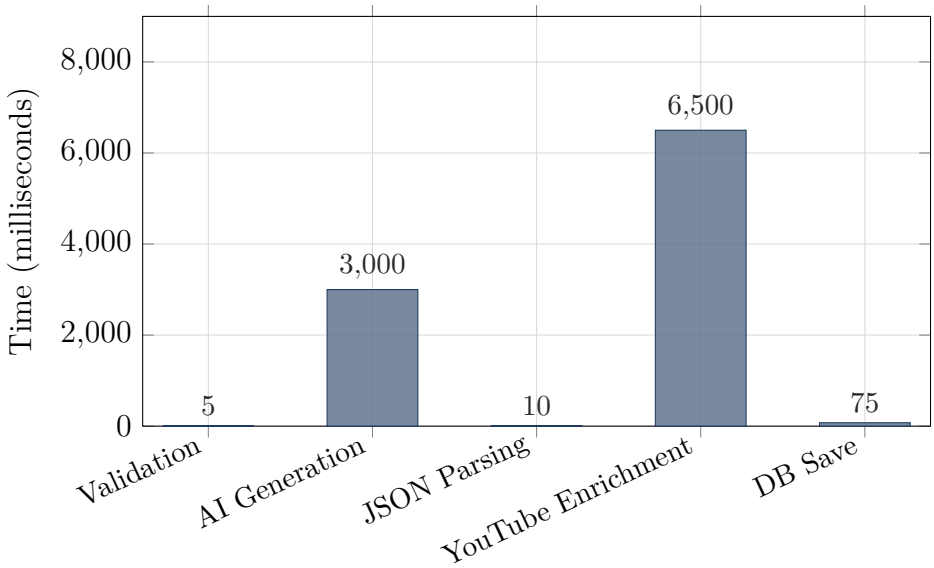

Figure 7.1: Course Generation Timing Breakdown — Average Duration per Phase (7-day course)

7.3 Groq vs. OpenAI Inference Speed

The project originally used the OpenAI GPT-3.5-turbo API but migrated to Groq for significantly faster inference. The Groq platform achieves these speeds through custom Language Processing Units (LPUs), which are application-specific integrated circuits optimized for sequential token generation with deterministic latency.

Table 7.3: AI Inference Speed Comparison

Provider	Avg. Latency	Tokens/sec	Cost (per 1M tokens)
OpenAI GPT-3.5-turbo	∼8,000 ms	∼50–80	$0.50
OpenAI GPT-4o-mini	∼6,000 ms	∼80–120	$0.15
Groq (LLaMA 3.1 70B)	∼2,500 ms	∼300–500	Free (rate-limited)
Groq (LLaMA 3.1 8B Instant)	∼1,200 ms	∼800+	Free (rate-limited)
Groq (Mixtral 8x7B)	∼1,800 ms	∼400–600	Free (rate-limited)

The migration from OpenAI to Groq reduced AI plan generation latency by approximately 3.2× (from ∼8 s to ∼2.5 s), which directly improved the overall course generation experience from ∼16 s to ∼10 s.

7.4 Database Performance

PostgreSQL is configured with several optimizations to ensure consistent performance under load.

7.4.1 Indexing Strategy

Table 7.4 documents all database indexes and their purpose.

Table 7.4: Database Index Configuration

Model	Field(s)	Type	Purpose
User	`email`	Unique	Login lookup by email
Course	`userId`	B-tree	Dashboard course list query
Course	`shareToken`	Unique	Shared course access
Progress	`courseId`	B-tree	Progress aggregation per course
Progress	`(courseId, day, lessonId)`	Unique	Upsert constraint for idempotent updates
ActivityLog	`userId`	B-tree	Heatmap query by user
ActivityLog	`date`	B-tree	Date-range filtering
ActivityLog	`(userId, date)`	Unique	Daily count upsert constraint
Note	`(courseId, day, lessonId)`	Unique	One note per lesson constraint

7.4.2 Query Performance

Key queries and their approximate execution characteristics:

- **Dashboard Load** (`findMany courses`): Uses the `userId` index, returns in <10 ms for up to 100 courses per user.
- **Course Detail** (`findFirst with planJson`): Single-row primary key lookup, <5 ms regardless of JSON size.
- **Progress Upsert**: Uses composite unique index, atomic upsert in <3 ms.
- **Activity Heatmap**: Date-range query with `userId` filter, returns ≤365 rows in <15 ms.

7.4.3 Connection Management

- **Connection Pooling**: Prisma manages a pool of database connections automatically, reusing connections across requests to minimize connection overhead.
- **Cascade Deletes**: Foreign key cascades (`onDelete: Cascade`) ensure that deleting a user removes all associated courses, progress records, activities, and notes without application-level cleanup logic.

- **Upsert Operations**: Progress and activity logging use atomic `upsert` operations to avoid race conditions when multiple tabs mark the same lesson complete simultaneously.

7.5 Frontend Performance

7.5.1 Build Optimization

- **Vite Build**: Production builds use Rollup with tree-shaking and code splitting for minimal bundle sizes. The final JavaScript bundle is approximately 280 KB gzipped.
- **Lazy Loading**: React Router supports lazy loading of page components to reduce initial load time. The `CreateCourse` and `CourseDetail` pages are code-split since they are not needed on initial load.
- **Asset Optimization**: Static assets (CSS, fonts) are fingerprinted for long-term caching with immutable cache headers.

7.5.2 Runtime Optimization

- **SSE for Feedback**: Server-Sent Events for course generation eliminate the need for polling, reducing unnecessary network traffic by ∼95% compared to a polling-based approach.
- **Thumbnail Optimization**: YouTube thumbnails use `medium` quality (320×180px) instead of high-resolution images to save bandwidth.
- **Skeleton Loading**: Animated skeleton components (`CourseSkeleton`) display immediately while data loads, preventing cumulative layout shift (CLS) and improving perceived performance.
- **Debounced State Updates**: Progress toggles use optimistic UI updates with server synchronization, so users see instant feedback.

7.5.3 Render Performance

- **Framer Motion**: Animations run on the compositor layer (`transform` and `opacity`) to avoid layout thrashing and maintain 60 fps.
- **Key-based Rendering**: Course cards and lesson lists use stable `key` props (UUIDs) to minimize unnecessary React re-renders.
- **Memoization**: Heavy computations (progress calculations, date formatting) use `useMemo` and `useCallback` hooks.

7.6 Scalability Considerations

Table 7.5 estimates system behavior under increasing user loads.

Table 7.5: Estimated Scalability Metrics

Metric	**10 Users**	**100 Users**	**1,000 Users**
Concurrent generations	2–3	20–30	200+ (needs queue)
DB connections needed	5	15	50+ (pool limit)
YouTube API quota/day	7 courses	7 courses	7 courses (shared)
Memory usage	∼80 MB	∼200 MB	∼1 GB

For scaling beyond ∼100 concurrent users, the recommended architecture changes include:

1. **Background Job Queue**: Move AI generation to a BullMQ worker process, returning a job ID to the frontend for status polling.
2. **Redis Caching**: Cache YouTube search results by keyword with a 24-hour TTL, reducing API quota consumption by an estimated 60–80%.
3. **Database Read Replicas**: Add PostgreSQL read replicas for dashboard and course detail queries.
4. **CDN**: Serve the React frontend from a CDN (Cloudflare, Vercel) for global edge caching.

Chapter 8

Challenges & Solutions

8.1 AI JSON Malformation

Challenge: Large language models frequently return JSON wrapped in markdown code blocks (`"‘json ... "‘`) or with trailing text outside the JSON structure, causing `JSON.parse()` to fail.

Solution: A multi-stage parsing pipeline:

1. Strip markdown code fences using regex: `content.replace(/"‘json\n?/g, ")`
2. Trim whitespace before parsing.
3. Validate the parsed object for required keys (`overview`, `modules`, `dailyPlan`).
4. Verify the number of days matches the user's request.
5. On failure, retry up to 3 times with exponential backoff.

8.2 AI Model Decommissioning

Challenge: Cloud AI providers periodically decommission models without notice, causing `400 Bad Request` errors.

Solution: The Groq service iterates through a prioritized list of models, automatically falling back to the next available model when one returns a decommissioning error or 400 status.

8.3 YouTube API Quota Limits

Challenge: The YouTube Data API v3 has a daily quota of 10,000 units. Each `search.list` call costs 100 units, and each `videos.list` call costs 1 unit. A single 7-day course with

14 lessons would consume $14 \times 100 + 14 \times 1 = 1{,}414$ units—allowing only ∼7 courses per day.

Solutions implemented:

- **Timeout per lesson**: 5-second timeout prevents slow searches from blocking the pipeline.
- **Graceful degradation**: If YouTube enrichment fails entirely, the course is returned without videos rather than failing.
- **Batch delays**: 100 ms delay between parallel batches reduces rate-limit pressure.
- **Future improvement**: Results caching (e.g., Redis) to avoid redundant searches for the same keywords.

8.4 Time-Budget Constraint Algorithm

Challenge: Selecting videos that fit a daily time budget while maximizing relevance is a constrained optimization problem. Naive approaches either exceeded the budget or selected low-quality videos.

Solution: A greedy algorithm that:

1. Filters out full courses, shorts, and excessively long videos.
2. Scores remaining videos by title match, keyword overlap, and educational indicators.
3. Sorts by relevance score (descending).
4. Iterates through sorted candidates, selecting the first that fits the remaining budget.
5. Falls back to the shortest relevant video if none fit, flagging the budget exceedance.

8.5 JWT Security Considerations

Challenge: Storing JWT tokens in `localStorage` is vulnerable to XSS attacks, but HttpOnly cookies introduce CSRF risks and complicate cross-origin deployments.

Solution: For the current deployment:

- Tokens are stored in `localStorage` with a 7-day expiration.
- Input validation on all endpoints prevents common injection vectors.
- CORS is configured to allow only the frontend origin.
- Prisma's parameterized queries prevent SQL injection.
- **Production recommendation**: Migrate to HttpOnly, Secure, SameSite cookies.

8.6 Server-Sent Events (SSE) Reliability

Challenge: SSE connections can drop due to proxy buffering (especially behind Nginx), timeouts, or client disconnection during course generation.

Solution:

- Set `X-Accel-Buffering: no` header to disable Nginx buffering.
- Set `Cache-Control: no-cache` and `Connection: keep-alive`.
- Wrap the entire SSE handler in try-catch with proper `res.end()` cleanup.
- Fallback: The non-SSE endpoint (`POST /api/course/generate`) remains available for clients that don't support SSE.

8.7 Cross-Origin Resource Sharing (CORS)

Challenge: During development, the React frontend runs on `localhost:5173` while the Express backend runs on `localhost:5000`, creating cross-origin requests that browsers block by default.

Solution:

- The `cors` middleware is configured with a whitelist of allowed origins read from the `CORS_ORIGIN` environment variable.
- Only `GET`, `POST`, `PUT`, `DELETE` methods are allowed, with `Authorization` and `Content-Type` headers explicitly permitted.
- Preflight `OPTIONS` requests are cached for 86,400 seconds (24 hours) to reduce overhead.
- In production, CORS is tightened to allow only the deployed frontend domain.

8.8 Handling Large AI Responses

Challenge: For courses with 30+ days, the AI-generated JSON can exceed 50,000 tokens, occasionally causing Groq API timeouts or truncated responses.

Solution:

- **Token estimation**: Before prompting, the system estimates the expected output size based on the number of days and adjusts the `max_tokens` parameter accordingly.
- **Temperature tuning**: A temperature of 0.7 is used for the primary model to balance creativity with structure adherence; the regeneration endpoint uses 0.8 for more variety.

- **Chunked generation**: For courses exceeding 14 days, a future improvement will generate in weekly chunks, merging results server-side.
- **Validation recovery**: If the response JSON is truncated, the system attempts to repair common truncation patterns (e.g., missing closing brackets) before failing.

Chapter 9

Security & Testing

9.1 Security Architecture

Security is implemented as a multi-layered defense strategy spanning the frontend, API layer, and database.

9.1.1 Authentication Security

- **Password Hashing**: All passwords are hashed using bcrypt with 10 rounds of salting (`bcrypt.hash(password, 10)`). The bcrypt algorithm is intentionally slow (∼100 ms per hash), making brute-force attacks computationally infeasible. Even if the database is compromised, passwords cannot be reversed.
- **JWT Tokens**: JSON Web Tokens are signed with a server-side secret key (`JWT_SECRET` environment variable) using the HS256 algorithm. Tokens contain only the `userId` claim—no sensitive data is embedded in the payload.
- **Token Expiration**: Tokens expire after 7 days, forcing periodic re-authentication. The frontend's Axios response interceptor detects expired tokens (401 status) and automatically redirects to the login page.

9.1.2 Input Validation

Every API endpoint validates incoming data before processing:

Table 9.1: Input Validation Rules Across API Endpoints

Endpoint	Validation Rules
`POST /auth/register`	Name required, email format validated, password ≥6 characters
`POST /auth/login`	Email and password required, user existence check
`POST /course/generate`	Topic non-empty, level ∈ {beginner, intermediate, advanced}, days ∈ [1, 365], timePerDay ∈ [15, 480]
`PUT /course/progress`	courseId (UUID format), day (positive integer), lessonId (string)
`POST /course/:id/share`	Course ownership verified via JWT userId

9.1.3 SQL Injection Prevention

Prisma ORM generates parameterized queries for all database interactions, completely eliminating SQL injection vulnerabilities. For example, the user lookup query:

```
// Prisma auto-parameterizes this query
const user = await prisma.user.findUnique({
  where: { email: email.toLowerCase() },
});
// Generated SQL: SELECT * FROM "User" WHERE "email" = $1
// Parameter: ['user@example.com']
```

Listing 9.1: Prisma Parameterized Query Example

9.1.4 Cross-Site Scripting (XSS) Prevention

- React's JSX auto-escapes all rendered values by default, preventing DOM-based XSS.
- User-generated content (course notes) is stored as plain text and rendered with `textContent` rather than `innerHTML`.
- The YouTube iframe embeds use the `sandbox` attribute to restrict JavaScript execution within the embedded player.

9.1.5 Rate Limiting & Error Handling

- The Express server uses centralized error handling via the `AppError` class, which standardizes error responses with appropriate HTTP status codes.
- The `asyncHandler` wrapper catches all promise rejections, preventing unhandled exceptions from crashing the server.
- API keys (`GROQ_API_KEY`, `YOUTUBE_API_KEY`) are stored in environment variables and never committed to version control (listed in `.gitignore`).

9.2 Testing Strategy

The application employs a multi-level testing approach to ensure reliability.

9.2.1 API Testing

Manual API testing is performed using tools such as Postman and cURL. Key test scenarios include:

Table 9.2: API Test Scenarios

Test Case	Method	Expected	Validates
Valid registration	POST	201	User created, JWT returned
Duplicate email	POST	409	Conflict error message
Invalid login	POST	401	Generic error (no info leak)
Generate course (valid)	POST	200	JSON plan with correct days
Generate (invalid level)	POST	400	Validation error
Access without JWT	GET	401	Unauthorized error
Access other user's course	GET	404	Not found (ownership check)
Progress upsert	PUT	200	Idempotent update
Delete course	DELETE	200	Cascade cleanup verified
Shared course (public)	GET	200	No auth required

9.2.2 Frontend Testing

- **Component Testing**: Individual React components are tested in isolation to verify correct rendering, prop handling, and state management.
- **Integration Testing**: End-to-end flows (registration → login → create course → view course → mark progress) are tested manually in the browser to verify the complete user journey.
- **Responsive Testing**: The application is tested across viewport sizes (320px mobile, 768px tablet, 1280px desktop) to verify the responsive grid layout of course cards and the collapsible navigation.
- **Dark Mode Testing**: All components are verified in both light and dark themes to ensure text contrast and readability.

9.2.3 AI Output Validation Testing

The AI-generated JSON output is validated against a rigorous set of structural requirements:

1. Presence of `overview`, `modules`, and `dailyPlan` top-level keys.
2. `dailyPlan` array length matches the requested number of days.

3. Each day contains a non-empty `lessons` array.
4. Each lesson has `title`, `description`, `timeMinutes`, and `importance` fields.
5. Quiz questions have exactly 4 options with a valid `correctAnswer` index.
6. Module references (`moduleId`) in the daily plan correspond to defined modules.

9.2.4 Error Recovery Testing

The retry and fallback mechanisms are tested by:

- Simulating API timeouts to verify exponential backoff behavior.
- Requesting models known to return 400 errors to verify multi-model fallback.
- Disconnecting network during SSE streaming to verify graceful frontend fallback.
- Exhausting YouTube API quota to verify graceful degradation (course returned without videos).

Chapter 10

Conclusion

10.1 Summary

PlanMyStudy AI demonstrates that generative AI can be practically applied to educational technology to solve the real problem of learning material curation and study planning. The system successfully:

- Generates structured, multi-day study plans from a single topic specification using Groq AI (LLaMA/Mixtral models) with reliable JSON output through prompt engineering and retry logic.
- Enriches each lesson with curated YouTube videos selected through a time-budget-aware algorithm that balances relevance, quality, and duration constraints.
- Provides a complete learning management experience with progress tracking, activity heatmaps, note-taking, quizzes, PDF export, and course sharing.
- Implements industry-standard security practices including JWT authentication, bcrypt password hashing, input validation, and parameterized database queries.
- Achieves responsive course generation times (8–12 seconds) through parallel YouTube fetching and Groq's ultra-fast AI inference.

10.2 Key Technical Achievements

1. **Multi-Model Fallback**: Automatic failover across three AI models ensures high availability despite model decommissioning.
2. **Time-Budget Algorithm**: The greedy video selection algorithm effectively solves a constrained optimization problem in real-time.
3. **Parallel API Orchestration**: Batched `Promise.all` calls provide a $\sim$3.3$\times$ speedup over sequential fetching while respecting rate limits.

4. **SSE Progress Streaming**: Real-time generation feedback significantly improves perceived performance and user experience.
5. **Hybrid Data Strategy**: Using JSON for dynamic content (course plans) and relational tables for structured data (users, progress, activity) combines the benefits of both approaches.

10.3 Lessons Learned

Several important lessons emerged during the development of PlanMyStudy AI:

1. **AI output is non-deterministic**: The same prompt can produce structurally different JSON responses across models and even across calls to the same model. Robust validation and retry logic is essential—not optional—when building production systems on LLM output.

2. **Third-party API quotas shape architecture**: The YouTube Data API's 10,000-unit daily quota became a hard constraint that forced architectural decisions (batching, result caching, graceful degradation). API quota analysis should be performed *before* committing to an integration, not after.

3. **Perceived performance matters as much as actual performance**: The SSE progress streaming feature was the most impactful UX improvement despite adding code complexity. Users tolerate a 10-second wait when they see progress updates, but abandon a 5-second wait with no feedback.

4. **Hybrid data models work**: Storing the AI-generated course plan as JSON while maintaining relational tables for user state (progress, notes, activity) proved to be an effective balance. Premature normalization would have added significant complexity without proportional benefit.

5. **Development velocity through ORMs**: Prisma significantly accelerated development by providing type-safe queries, automatic migrations, and eliminating entire classes of bugs (SQL injection, type mismatches).

10.4 Comparison with Existing Platforms

PlanMyStudy AI differentiates itself from existing learning platforms in several ways:

Table 10.1: Feature Comparison with Existing Platforms

Feature	PlanMyStudy AI	Coursera	Khan Academy	YouTube
AI-generated syllabus		—	—	—
Custom time budget		—	—	—
Auto-curated videos		—	—	
Progress tracking				—
Quizzes				—
Free & open-source		—		
Personalized day-by-day plan		—	—	—
Note-taking per lesson		—	—	—
PDF export		—	—	—

10.5 Future Scope

1. **Caching Layer**: Add Redis caching for YouTube search results (24-hour TTL) and frequently generated course plans. Expected to reduce YouTube API quota usage by 60–80% and improve response times for repeat topics by $\sim 5\times$.

2. **Real-Time Collaboration**: WebSocket-based study groups enabling multiple users to follow the same course, share notes, and see each other's progress in real time.

3. **Mobile Application**: A React Native version for iOS and Android, sharing the API layer with the web application and adding push notification support for study reminders.

4. **Advanced Analytics**: A learning analytics dashboard with insights such as optimal study times, topic mastery curves, and spaced-repetition scheduling to maximize long-term retention.

5. **Multi-Modal Resources**: Integration with additional content types including articles (Medium, Dev.to), textbooks (Open Library API), interactive coding exercises (JDoodle API), and podcast episodes.

6. **AI Study Assistant**: A conversational chatbot embedded in the course detail page, capable of answering questions about the current lesson's content using the course plan as context.

7. **Email Notifications**: An automated email system using Nodemailer with daily study reminders, weekly progress reports, and streak recovery notifications.
8. **Offline Support**: Progressive Web App (PWA) implementation with service workers for offline access to previously loaded course plans, enabling study in low-connectivity environments.
9. **Horizontal Scaling**: Queue-based background processing using BullMQ with Redis, decoupling the course generation from the HTTP request cycle. This enables load balancing across multiple backend instances and prevents timeout issues for long-running generations.
10. **Gamification**: Achievement badges, experience points, leaderboards, and streak rewards to increase user engagement and long-term retention.

Appendix A

Backend Code Listings

A.1 Authentication Controller (authController.js)

```
import bcrypt from 'bcryptjs';
import prisma from '../config/database.js';
import { generateToken } from '../utils/jwt.js';
import { AppError, asyncHandler } from '../utils/errors.js';

// Register a new user
export const register = asyncHandler(async (req, res, next) => {
  const { name, email, password } = req.body;

  if (!name || !email || !password) {
    throw new AppError('Name, email, and password are required',
   400);
  }
  if (password.length < 6) {
    throw new AppError('Password must be at least 6 characters',
   400);
  }

  const existingUser = await prisma.user.findUnique({
    where: { email: email.toLowerCase() },
  });
  if (existingUser) {
    throw new AppError('User with this email already exists', 409);
  }

  const hashedPassword = await bcrypt.hash(password, 10);
  const user = await prisma.user.create({
    data: { name, email: email.toLowerCase(), password:
   hashedPassword },
```

```
    select: { id: true, name: true, email: true, createdAt: true },
  });

  const token = generateToken(user.id);
  res.status(201).json({ success: true, data: { user, token } });
});

// Login user
export const login = asyncHandler(async (req, res, next) => {
  const { email, password } = req.body;
  if (!email || !password) {
    throw new AppError('Email and password are required', 400);
  }

  const user = await prisma.user.findUnique({
    where: { email: email.toLowerCase() },
  });
  if (!user) throw new AppError('Invalid email or password', 401);

  const isPasswordValid = await bcrypt.compare(password, user.
   password);
  if (!isPasswordValid) throw new AppError('Invalid email or
   password', 401);

  const token = generateToken(user.id);
  res.json({
    success: true,
    data: {
      user: { id: user.id, name: user.name, email: user.email },
      token,
    },
  });
});
```

Listing A.1: authController.js – User Registration and Login

A.2 JWT Utilities (jwt.js)

```
import jwt from 'jsonwebtoken';
import env from '../config/env.js';

export const generateToken = (userId) => {
  return jwt.sign({ userId }, env.jwtSecret, {
    expiresIn: '7d',
  });
```

```
};

export const verifyToken = (token) => {
  try {
    return jwt.verify(token, env.jwtSecret);
  } catch (error) {
    throw new Error('Invalid or expired token');
  }
};
```

Listing A.2: jwt.js – Token Generation and Verification

A.3 Authentication Middleware (auth.js)

```
import { verifyToken } from '../utils/jwt.js';
import { AppError } from '../utils/errors.js';

export const authenticate = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      throw new AppError('No token provided', 401);
    }
    const token = authHeader.substring(7);
    const decoded = verifyToken(token);
    req.userId = decoded.userId;
    next();
  } catch (error) {
    if (error instanceof AppError) return next(error);
    next(new AppError('Authentication failed', 401));
  }
};
```

Listing A.3: auth.js – JWT Authentication Middleware

A.4 Course Controller — Generate Course (excerpt)

```
export const generateCourse = asyncHandler(async (req, res, next) =>
    {
  const { topic, level, days, timePerDay } = req.body;
  const userId = req.userId;

  // Validate input
  if (!topic || !level || !days || !timePerDay) {
```

```
    throw new AppError('Topic, level, days, and timePerDay are required', 400);
  }
  const validLevels = ['beginner', 'intermediate', 'advanced'];
  if (!validLevels.includes(level.toLowerCase())) {
    throw new AppError('Level must be beginner, intermediate, or advanced', 400);
  }

  // Generate course plan using AI
  let coursePlan = await generateCoursePlan(topic, level, days, timePerDay);

  // Enrich with YouTube videos (respects time budget)
  try {
    coursePlan = await Promise.race([
      enrichCourseWithVideos(coursePlan, timePerDay),
      new Promise((resolve) => setTimeout(() => resolve(coursePlan), 30000)),
    ]);
  } catch (error) {
    console.log('YouTube enrichment skipped:', error.message);
  }

  // Save course to database
  const course = await prisma.course.create({
    data: {
      userId, topic, level: level.toLowerCase(),
      days, timePerDay,
      planJson: JSON.stringify(coursePlan),
    },
  });

  res.status(201).json({
    success: true,
    data: { course: { id: course.id, topic, level, days, timePerDay,
      plan: coursePlan, createdAt: course.createdAt } },
  });
});
```

Listing A.4: courseController.js – Course Generation Endpoint

Appendix B

Sample API Payloads

B.1 AI Course Generation Request

```
{
  "topic": "React Development",
  "level": "beginner",
  "days": 7,
  "timePerDay": 60
}
```

Listing B.1: Sample POST /api/course/generate Request Body

B.2 AI Course Generation Response (Excerpt)

```
{
  "overview": {
    "title": "React Development Fundamentals",
    "description": "A comprehensive 7-day beginner course...",
    "objectives": [
      "Understand React component architecture",
      "Master JSX syntax and expressions",
      "Build interactive UIs with state and props"
    ]
  },
  "modules": [
    {
      "id": "m1",
      "title": "React Basics",
      "description": "Introduction to React concepts",
      "objectives": ["Understand JSX", "Create components"]
```

```
      }
    ],
    "dailyPlan": [
      {
        "day": 1,
        "moduleId": "m1",
        "lessons": [
          {
            "id": "l1",
            "title": "Introduction to React",
            "description": "What is React and why use it",
            "objectives": ["Understand SPA architecture"],
            "timeMinutes": 30,
            "keywords": ["react", "SPA", "virtual DOM"],
            "importance": "core",
            "quiz": [
              {
                "question": "What is the Virtual DOM?",
                "options": [
                  "A) A lightweight copy of the real DOM",
                  "B) A database",
                  "C) A CSS framework",
                  "D) A server"
                ],
                "correctAnswer": "A",
                "explanation": "The Virtual DOM is a lightweight
                  JavaScript representation of the real DOM..."
              }
            ]
          }
        ]
      }
    ]
  }
```

Listing B.2: Groq AI Response Structure (abbreviated)

B.3 YouTube API Response (Excerpt)

```
{
  "kind": "youtube#videoListResponse",
  "items": [
    {
      "id": "dGcsHMXbSOA",
      "snippet": {
```

```
7          "title": "React Tutorial for Beginners",
8          "description": "Learn React.js in this beginner tutorial..."
      ,
9          "channelTitle": "Programming with Mosh",
10         "thumbnails": {
11           "medium": {
12             "url": "https://i.ytimg.com/vi/dGcsHMXbSOA/mqdefault.jpg
      ",
13             "width": 320,
14             "height": 180
15           }
16         },
17         "publishedAt": "2023-06-15T14:00:00Z"
18       },
19       "contentDetails": {
20         "duration": "PT28M45S"
21       },
22       "statistics": {
23         "viewCount": "1250000"
24       }
25     }
26   ]
27 }
```


Listing B.3: YouTube Data API v3 – Video Details Response (abbreviated)

Appendix C

Glossary

Term	Definition
API	Application Programming Interface — a set of protocols for building and integrating application software.
bcrypt	A password-hashing algorithm based on the Blowfish cipher, designed to be computationally expensive to resist brute-force attacks.
CORS	Cross-Origin Resource Sharing — an HTTP-header-based mechanism allowing servers to indicate permitted origins for browser requests.
Express	A minimal and flexible Node.js web application framework providing a robust set of features for web and mobile applications.
Groq	An AI inference platform using custom Language Processing Units (LPUs) for ultra-fast LLM inference.
ISO 8601	An international standard for date and time representations. YouTube uses the duration component (e.g., `PT4M13S`).
JSON	JavaScript Object Notation — a lightweight data-interchange format.
JWT	JSON Web Token — a compact, URL-safe means of representing claims between two parties, defined in RFC 7519.
LLaMA	Large Language Model Meta AI — a family of open-weight language models released by Meta.

Term	Definition
LLM	Large Language Model — a neural network model trained on large text corpora for natural language tasks.
Middleware	Software that acts as an intermediary between an OS or database and applications, or between request and response in web frameworks.
Mixtral	A mixture-of-experts large language model by Mistral AI, combining multiple expert sub-networks.
Node.js	A JavaScript runtime built on Chrome's V8 engine for building scalable server-side applications.
ORM	Object-Relational Mapping — a technique for converting data between incompatible type systems in object-oriented programming.
PostgreSQL	An advanced open-source relational database management system emphasizing extensibility and SQL compliance.
Prisma	A next-generation Node.js and TypeScript ORM providing type-safe database access with auto-generated queries.
React	A JavaScript library for building user interfaces, maintained by Meta, using a component-based architecture.
REST	Representational State Transfer — an architectural style for distributed hypermedia systems, commonly used for web APIs.
SPA	Single-Page Application — a web application that loads a single HTML page and dynamically updates content.
SSE	Server-Sent Events — a server push technology enabling the server to send real-time updates to the browser via HTTP.
UUID	Universally Unique Identifier — a 128-bit label used for identification in distributed systems.
Vite	A modern frontend build tool providing fast Hot Module Replacement (HMR) and optimized production builds.
XSS	Cross-Site Scripting — a security vulnerability enabling attackers to inject malicious scripts into web pages.
YouTube Data API	Google's API for accessing YouTube video data, including search, metadata, and content details.

References

[1] Groq, "Groq API Documentation," 2024. [Online]. Available: https://console.groq.com/docs

[2] Google, "YouTube Data API v3 Documentation," 2024. [Online]. Available: https://developers.google.com/youtube/v3

[3] Meta, "React Documentation," 2024. [Online]. Available: https://react.dev/

[4] Prisma, "Prisma ORM Documentation," 2024. [Online]. Available: https://www.prisma.io/docs

[5] Express.js, "Express Documentation," 2024. [Online]. Available: https://expressjs.com/

[6] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519

[7] Node.js, "Node.js Documentation," 2024. [Online]. Available: https://nodejs.org/docs

[8] PostgreSQL Global Development Group, "PostgreSQL Documentation," 2024. [Online]. Available: https://www.postgresql.org/docs/

[9] Vite, "Vite Documentation," 2024. [Online]. Available: https://vitejs.dev/

[10] Tailwind Labs, "Tailwind CSS Documentation," 2024. [Online]. Available: https://tailwindcss.com/docs

[11] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in *Proc. USENIX Annual Technical Conference*, 1999.

[12] I. Hickson, "Server-Sent Events," W3C Recommendation, Feb. 2015. [Online]. Available: https://www.w3.org/TR/eventsource/